UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

Experiment 4

Student Name: Prateek Verma | **UID: 25MCC20062**
Branch: MCA (CC & DEVOPS) | **Section/Group: 25MCC-101/A**
Semester: IInd | **Date of Performance: 03/02/2026**
Subject Name: Technical Training | **Subject Code: 25CAP-652**

Title: Implementation of Iterative Control Structures using FOR, WHILE, and LOOP in PostgreSQL

1. **Aim:**
   a. To understand and implement iterative control structures in PostgreSQL conceptually, including FOR loops, WHILE loops, and basic LOOP constructs, for repeated execution of database logic.

2. **Software Requirements**:
   a. PostgreSQL (Database Server)
   b. pgAdmin
   c. OS: Windows

3. **Objective:**
   a. To understand why iteration is required in database programming
   b. To learn the purpose and behavior of FOR, WHILE, and LOOP constructs
   c. To understand how repeated data processing is handled in databases
   d. To relate loop concepts to real-world batch processing scenarios
   e. To strengthen conceptual knowledge of procedural SQL used in enterprise systems.

4. **Procedure of the Practical:**
   a. Start the system.
   b. Open pgAdmin.
   c. Create and select the database in which you want to perform the experiment.
   d. Establish connection to the database using Alt+Shift+Q.
   e. Run the following queries given in Experiment Steps (5).

5. Experiment Steps:

a. For Loop (simple iteration):

```
DO $$
BEGIN
 FOR i IN 1..5 LOOP
  RAISE NOTICE 'Number: %', i;
 END LOOP;
END $$;
```

```
NOTICE:  Number: 1
NOTICE:  Number: 2
NOTICE:  Number: 3
NOTICE:  Number: 4
NOTICE:  Number: 5
DO
```

b. For Loop with Query (Row-by-Row Processing):

```
DO $$
DECLARE
        rec RECORD;
BEGIN
        FOR rec in SELECT id, marks FROM "Experiment_3".students LOOP
                RAISE NOTICE 'ID: %, Marks: %',rec.id, rec.marks;
        END LOOP;
END $$;
```

```
NOTICE:  ID: 1, Marks: 95
NOTICE:  ID: 2, Marks: 82
NOTICE:  ID: 3, Marks: 68
NOTICE:  ID: 4, Marks: 54
NOTICE:  ID: 5, Marks: 40
NOTICE:  ID: 6, Marks: 25
DO
```

c. While Loop (conditional iteration):

```
DO $$
DECLARE
        i int:=1;
BEGIN
        WHILE i<=5 LOOP
                RAISE NOTICE 'WHILE LOOP turn %',i;
                i:=i+1;
        END LOOP;
END $$
```

```
NOTICE:  WHILE LOOP turn 1
NOTICE:  WHILE LOOP turn 2
NOTICE:  WHILE LOOP turn 3
NOTICE:  WHILE LOOP turn 4
NOTICE:  WHILE LOOP turn 5
DO
```

d. Loop with 'EXIT WHEN':

```
DO $$
DECLARE
        i int :=1;
BEGIN
        LOOP
                RAISE NOTICE 'i=%',i;
                i:=i+1;
                EXIT WHEN i>5;
        END LOOP;
END $$;
```

```
NOTICE:  i=1
NOTICE:  i=2
NOTICE:  i=3
NOTICE:  i=4
NOTICE:  i=5
DO
```

e. Salary increment using For Loop:

```
DO $$
DECLARE
        rec RECORD;
BEGIN
 FOR rec IN (SELECT id, salary from salaries)
 LOOP
   -- Update logic
   UPDATE salaries
   SET salary = rec.salary * 1.10
   WHERE id = rec.id;
 END LOOP;
END $$;
```

Before:

	id [PK] integer	salary numeric (10,2)
1	1	35000.00
2	2	55000.00
3	3	45000.00

After:

	id [PK] integer	salary numeric (10,2)
1	1	38500.00
2	2	60500.00
3	3	49500.00

f. Combining Loop with IF condition:

```
DO $$
DECLARE
    rec RECORD;
BEGIN
    For rec in (select id, marks from "Experiment_3".students)
    LOOP
        IF rec.marks>50 THEN
            RAISE NOTICE 'ID: %, Marks: %',rec.id,rec.marks;
        END IF;
    END LOOP;
END $$;
```

```
NOTICE:  ID: 1, Marks: 95
NOTICE:  ID: 2, Marks: 82
NOTICE:  ID: 3, Marks: 68
NOTICE:  ID: 4, Marks: 54
DO
```

6. Input/Output Details:

a. The input for the experiment is the SQL queries mentioned in Experiment Steps (5).

b. The output for each query is attached after the query itself.

7. Learning Outcome

a. Understanding of how iterative control structures work in PostgreSQL at a conceptual level.
b. Usage of loops in database systems, such as workflow engines, complex decision cycles, validation loops, etc.
c. Foundational knowledge required for writing procedural logic in enterprise-grade applications.